

## Patrones de Diseño

### Arquitectura en Capas vs Modelo Vista Controlador (MVC)

Cuando hablamos de organizar el código de un sistema, dos de las formas más comunes son la arquitectura en capas y el modelo MVC (Modelo-Vista-Controlador). Aunque pueden parecer similares porque ambas separan responsabilidades, en realidad tienen enfoques distintos.

#### CAPAS

La arquitectura en capas organiza el sistema como si fuera una “torre” dividida en niveles. Cada capa tiene una función específica y solo se comunica con la capa que tiene debajo o encima.

Se divide en:

- **Presentación** → lo que ve el usuario (pantallas)
- **Lógica de negocio** → donde se toman decisiones (reglas del sistema)
- **Acceso a datos** → conexión con la base de datos

Ejemplo:

Un sistema de gestión de estudiantes:

- La capa de presentación muestra el formulario para registrar alumnos
- La lógica valida que los datos estén correctos
- La capa de datos guarda la información en la base de datos

Cada capa hace su parte y pasa el resultado a la siguiente.

#### Estructura Típica de Archivos en MVC (ejemplo):

/app

/models (ej. Usuario.php)

/views (ej. perfil.html)

/controllers (ej. UsuarioController.php)

/public (índice, estilos, JS)

#### HEXAGONAL

El modelo MVC también separa responsabilidades, pero lo hace pensando más en cómo interactúa el usuario con el sistema, especialmente en aplicaciones web.

Se divide en:

- **Modelo** → maneja los datos y la lógica
- **Vista** → muestra la información al usuario
- **Controlador** → recibe las acciones del usuario y coordina todo

Ejemplo:

En una página web de registro:

- El usuario llena un formulario (Vista)
- El controlador recibe los datos al hacer clic en “guardar”
- El modelo procesa y guarda la información

Aquí el controlador es como un “intermediario” que conecta todo.

APLICAdo al proyecto

El **núcleo de dominio** es Python puro, sin ningún `import django`. Ahí viven tus entidades (clases de datos) y tus casos de uso (la lógica real del sistema). Este código es 100% testeable sin levantar un servidor ni una base de datos.

Los **puertos** son interfaces abstractas (en Python, clases con `ABC`) que el dominio define. Por ejemplo, `IUsuarioRepositorio` con un método `guardar(usuario)`. El dominio no sabe cómo se implementa ese método.

Los **adaptadores de entrada** (izquierda del hexágono) son las DRF Views. Su único trabajo es traducir una petición HTTP en una llamada al caso de uso correcto. No tienen lógica de negocio.

Los **adaptadores de salida** (derecha) son las implementaciones concretas: `DjangoUsuarioRepositorio` que implementa `IUsuarioRepositorio` usando el ORM de Django. Si mañana cambias de PostgreSQL a MongoDB, solo cambias este adaptador — el dominio no se toca.

**Ventaja clave con tu Git Flow:** cada rama de feature puede trabajar en adaptadores sin romper el dominio. Puedes incluso usar mocks en `develop` mientras el adaptador real de base de datos se desarrolla en otra rama.

Django no está diseñado para hexagonal nativamente. Tendrás que ir en contra de algunos convenios del framework, especialmente la tentación de poner lógica en los modelos o en las vistas. Requiere disciplina del equipo.

## Comparación entre ambas

Aunque ambas buscan ordenar el código, la diferencia principal está en su enfoque:

- La arquitectura en capas organiza el sistema por niveles técnicos (presentación, lógica, datos).
- MVC organiza el sistema según la interacción del usuario (vista, control y datos). En otras palabras:
- Capas = estructura interna del sistema
- MVC = cómo fluye la interacción con el usuario

## Domain-Driven Design

Domain-Driven Design (DDD) es un enfoque de desarrollo de software que coloca el **dominio del negocio** en el centro del diseño, permitiendo organizar mejor el sistema y hacerlo más entendible y mantenible. Se divide en dos tipos de patrones: estratégicos y tácticos.

### Patrones estratégicos:

- **Lenguaje ubicuo (Ubiquitous Language):** vocabulario compartido entre desarrolladores y expertos del dominio, usado de forma consistente en código, documentación y comunicación. Por ejemplo, si en el sistema se usa “Estudiante”, ese mismo término se usa en todo el proyecto.
- **Contexto delimitado (Bounded Context):** define los límites donde un modelo tiene un significado específico. Por ejemplo, “Usuario” puede significar algo distinto en el módulo académico que en el administrativo.
- **Mapa de contextos (Context Map):** representación de cómo se relacionan los diferentes contextos del sistema y cómo intercambian información. Por ejemplo, el módulo de “Usuarios” gestiona la autenticación, el módulo “Académico” utiliza esos datos para identificar estudiantes, y el módulo “Pagos” usa la información académica para procesar matrículas.
- **Subdominios (Subdomain):** división del sistema en partes:
  - **Core Domain:** parte principal del sistema (ej: gestión académica).
  - **Supporting Domain:** funciones de apoyo (ej: reportes).
  - **Generic Domain:** funcionalidades comunes (ej: autenticación o envío de correos).

### Patrones tácticos:

- **Entidad (Entity):** objeto con identidad única que se mantiene en el tiempo. Ejemplo: un estudiante con su ID, aunque cambien sus datos.
- **Objeto de valor (Value Object):** objeto sin identidad propia, definido solo por sus atributos. Ejemplo: una dirección o una calificación.
- **Agregado (Aggregate):** conjunto de objetos relacionados que se manejan como una unidad. Ejemplo: un estudiante junto con sus materias y notas.
- **Raíz del agregado (Aggregate Root):** entidad principal que controla el acceso a los demás objetos del agregado. Ejemplo: el estudiante controla sus calificaciones.
- **Repositorio (Repository):** permite guardar y recuperar datos del sistema, ocultando el acceso directo a la base de datos.
- **Servicio de dominio (Domain Service):** contiene lógica de negocio que no pertenece a una entidad específica. Ejemplo: calcular el promedio de notas de un estudiante.
- **Evento de dominio (Domain Event):** representa un suceso importante dentro del sistema. Ejemplo: “estudiante registrado” o “nota actualizada”.
- **Factoría (Factory):** se encarga de crear objetos complejos cuando su construcción no es simple. Por ejemplo, crear un estudiante con usuario, contraseña, rol y datos iniciales correctamente configurados en un solo proceso.

En DDD no hay una estructura única obligatoria, pero si hay una forma típica de organizar carpetas basada al dominio:

Estructura en DDD:

/proyecto

  /domain

    /entities       (ej: Estudiante.py)

    /value\_objects   (ej: Nota.py)

    /aggregates      (lógica de agrupación)

    /repositories    (interfaces de acceso a datos)

    /services        (lógica de negocio)

    /events          (eventos del dominio)

  /application

    /use\_cases       (casos de uso, ej: registrar\_estudiante.py)

  /infrastructure

    /database        (configuración de base de datos)

    /repositories    (implementaciones reales)

  /interfaces

    /api             (controladores, endpoints)

## DIFERENCIAS ENTRE MVC Y POR CAPAS (EN CUANTO A ARCHIVOS)

mvc

```
sistema_academico/  
  notas/ - app Django  
    models.py - Alumno, Nota, Materia, Padre  
    views.py - ⚠ aquí vive TODO: lógica + HTTP  
    serializers.py - convierte objetos a JSON  
    urls.py - /notas/alumno/{id}/  
    admin.py  
    tests.py - difícil de testear sin BD  
  config/  
    settings.py  
    urls.py  
  manage.py
```

## por capas

sistema\_academico/

notas/ — app Django

— capa presentación —

views.py — solo recibe HTTP, llama al service

serializers.py — valida entrada y formatea salida

urls.py — /api/notas/alumno/{id}/

— capa servicios (negocio) —

services.py — ¿puede el padre ver estas notas?

    calcular\_promedio(), verificar\_acceso()

— capa datos —

models.py — Alumno, Nota, Materia, Padre

selectors.py — queries reutilizables al ORM

admin.py

tests/

    test\_services.py — testea negocio sin HTTP

    test\_views.py — testea endpoints

config/

    settings/

        base.py dev.py prod.py

manage.py

| Tabla comparativa MVC y arquitectura en capas aplicada al sistema académico                                                  |                                                                            |                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Criterio                                                                                                                     | MVC / MTV (Django clásico)                                                 | Arquitectura en capas                                                                                 |
| Número de capas                                                                                                              | 3 — Model, View (controlador), Template                                    | 4 — Presentación, Servicios, Repositorio, Datos                                                       |
| ¿Dónde va la lógica de negocio?                                                                                              | En <code>views.py</code> junto con el manejo HTTP                          | En <code>services.py</code> aislada y reutilizable                                                    |
| Ejemplo en el sistema académico                                                                                              | La vista verifica si el padre puede ver las notas Y arma la respuesta JSON | La vista solo llama a <code>NotasService.obtener()</code> ; el service hace la verificación           |
| Testabilidad                                                                                                                 | Media — hay que simular peticiones HTTP para probar la lógica              | Alta — <code>services.py</code> se prueba con Python puro, sin levantar servidor                      |
| Curva de aprendizaje                                                                                                         | Baja — sigue los convenios de Django directamente                          | Media — requiere disciplina para no mezclar capas                                                     |
| Velocidad inicial                                                                                                            | Rápida — ideal para prototipos y MVPs                                      | Más lenta — hay que definir la estructura antes de avanzar                                            |
| Escalabilidad                                                                                                                | Limitada — las vistas crecen y se vuelven difíciles de mantener            | Alta — cada capa crece de forma independiente                                                         |
| Trabajo en equipo (Git Flow)                                                                                                 | Conflictos frecuentes — varios devs editan el mismo <code>views.py</code>  | Menos conflictos — uno trabaja en <code>services</code> , otro en <code>serializers</code>            |
| Reutilización del código                                                                                                     | Baja — lógica atada a la petición HTTP                                     | Alta — <code>services</code> se llama desde vistas, comandos, tareas Celery                           |
| Archivos clave                                                                                                               | <code>models.py</code> , <code>views.py</code> , <code>urls.py</code>      | <code>models.py</code> , <code>services.py</code> , <code>selectors.py</code> , <code>views.py</code> |
| Cuándo usarlo                                                                                                                | Proyectos pequeños, equipo de 1–2 personas, plazos cortos                  | Proyectos que van a crecer, equipos de 3+ personas, lógica compleja                                   |
| Riesgo principal                                                                                                             | Fat views — vistas de 200+ líneas mezclando todo                           | Over-engineering — crear capas innecesarias en proyectos simples                                      |
| <div>¿Cómo evitar fat views? ↗</div> <div>Ver <code>services.py</code> completo ↗</div> <div>¿Cuándo usar hexagonal? ↗</div> |                                                                            |                                                                                                       |